FilsPay Submission

Team: [Team name]

Member	Owned
[Name]	[e.g. settlement & money fixes]
[Name]	[e.g. deployment & load test]
[Name]	[e.g. UI, slides]

Live URLs

Checkout	[https://…/pay/<token>]
Merchant dashboard	[https://…]
Admin console	[https://…]
Public API	[https://…/healthz]

Repository

Public fork: **[https://github.com/aliabdulla5/fils-pay — confirm this is your public fork]**
Last commit before the deadline: **[commit id]**

1. Summary

What we found. The case file's 25 items split into 74 distinct defects. A second review pass found 18 more (inventory items 75–92). In total that is 92 findings in `docs/BUG_INVENTORY.md`: 79 confirmed and fixed, 10 confirmed and left open (most by explicit instruction, the rest needing DBA or design approval), and 1 rejected as intentional. The new ones include an unauthenticated reporting endpoint and a cross-merchant data leak. On the money side, a hold-expiry script deleted every hold on a quiet night, a capture replay double-credited a merchant's discount, and an admin re-run could close an unfinished business day and strand that day's later captures.

What we fixed. Every confirmed defect that the repository rules allow us to fix, each with the smallest change that works. Fixes are in the source and migrations; no test was edited.

What we left alone. Frozen code (`packages/money`, the Go money mirror), locked tests, the v1 partner API's 501 stubs, the deferred refund webhook, the documented `Math.random()` and display-name rules, and anything needing a DBA-approved migration (widening the platform volume counter, replacing the REAL lifetime estimate).

Main root cause. The same fact is computed in two places that drifted apart, and nothing checks that they agree. Examples:

- The balance button versus the ledger walk.
- Capture-time fees versus settlement fees.
- The analytics rebuild versus the reporting worker.
- Webhook delivery being used to write a payout record.
- The "docs are truth" agent files contradicting the ADRs.

Locked tests that assert the buggy behaviour make the drift look correct.

2. Architecture and deployment

```
Internet
   │  HTTPS (reverse proxy / TLS)   [your proxy: Caddy/nginx — fill in]
   ├─► web  (checkout + merchant dashboard, static build)        [port]
   ├─► admin (ops console, static build)                         [port]
   └─► api  :3000   Fastify — merchant, checkout, admin, acquirer callbacks
              │ webhook dispatcher runs in-process (LISTEN/NOTIFY + poll)
              ▼
         postgres :5432 (bound to 127.0.0.1 only)
              ▲            ▲
   settlement :8091 (internal only, bearer token)   reporting (no port; outbox consumer)
   daemon: daily close 00:15 Asia/Bahrain
```

- **Redeploy:** `docker compose -f ops/docker-compose.yml up -d --build api settlement reporting`, with migrations run first: `docker compose -f ops/docker-compose.yml run --rm migrate`. The `seed` job sits behind a profile and never runs on `up`, because it truncates the ledger.
- **Secrets:** an `ops/.env` file that is gitignored, read by compose for `${VAR}` substitution (template: `ops/.env.docker.example`). `NODE_ENV` defaults to `production`, and production refuses to boot without `SETTLEMENT_INTERNAL_TOKEN`, `ACQUIRER_CALLBACK_SIGNING_SECRET` and

`WEBHOOK_SECRET_ENCRYPTION_KEY`. One acquirer signing secret was committed to git history in the past and must be rotated with the acquirer.

- **Health checks:** the API serves `/healthz` (liveness) and `/readyz` (DB reachable), and Postgres has a compose healthcheck. **[Add your restart policy (e.g. `restart: unless-stopped`) and API limits (0.5 vCPU / 512 MB) if your deployment sets them.]**
- **Redeploys and settlement runs:** each batch close and each payout is one Postgres transaction. On SIGTERM the daemon cancels its context, so an in-flight transaction commits or rolls back and never half-applies. Close and pay are idempotent per (merchant, date), so the next daemon tick, or `settlement run-batch --date=…`, completes the day safely. A run-batch for an unfinished business day is now refused (finding 78).

3. Capacity

[Fill in from your own load test on the deployed server: tool, scenario, users, p95 latency, error rate, graph.]

Estimate from the code, not yet measured. A checkout holds one pooled connection for the whole pay attempt: an advisory lock, then authorize, then capture, which keeps the row lock across the simulated acquirer call. The simulated acquirer takes 100–400 ms per call, so a checkout takes roughly 0.3–0.9 s. With `PG_POOL_MAX=5`, about 5 checkouts run at once, which is roughly 6–15 checkouts per second before requests queue on the pool. Raising the pool size is the first lever, within Postgres's connection budget on a 2 GB host.

Bottleneck: the API's database pool and the lock held across the acquirer call. **Overload behaviour today:** requests wait up to `PG_POOL_CONNECTION_TIMEOUT_MS` (5 s) for a connection, then fail with 500. There is no admission control or 429 yet. **[Describe what you added, e.g. proxy rate limits or a queue-length cap.]** Recovery: the pool and the process are stateless apart from Postgres, so a restart recovers without losing data (idempotency keys make client retries safe).

4. Findings report

The full inventory, with all 92 entries and their status, is `docs/BUG_INVENTORY.md`. Items 1–74 come from the first rounds (the sibling review); they are summarised in the table at the end of this section. Below are the 18 findings from the second review pass, written out in full.

Method for every entry: we traced the path from input to side effect, checked the ADRs, handover and changelog for intended behaviour, reproduced the defect, applied the smallest fix, and verified it against a freshly migrated Postgres 16 database. We used an untouched copy of `HEAD` plus only our patch, so unrelated working-tree changes could not affect the results. The full suites were then re-run: no new failures (see Appendix).

F75 — Reporting endpoint with no authentication

Symptom: `GET /v1/reporting/export-summary` returned 500 for everyone. **Found/reproduced:** route map review; `app.inject` without credentials returned 500. **Root cause:** the route was registered without the `requireMerchantAuth` preHandler, so `request.merchantAuth!` was undefined. **Fix:** preHandler added. **Verified:** unauthenticated returns 401, authenticated returns 200. **Existing data:** none.

F76 — Cross-merchant transaction report (IDOR)

Symptom: any merchant could read a marketplace sub-merchant's transactions by putting `{"merchantId": "<child>"}` in the report filter. **Root cause:** `resolveReportMerchantId` checked only that the id was *some* parent's split child, not the caller's. **Fix:** require `parent_merchant_id = caller`. **Verified:** an unrelated merchant resolves to itself; the real parent still reaches its child. **Existing data:** none; audit access logs if exposure matters.

F77 — Payout disbursement record written by webhook delivery

Symptom: `settlement_payout_disbursements` got one row per active webhook endpoint and another per replay or archive restore, and none for merchants without an endpoint. **Root cause:** the record was a "pre-delivery effect" of the first `settlement.paid` delivery (attempt 0, which every replay starts at), contrary to migration 0057 ("written inside PayoutBatch's transaction, one row per paid batch"). **Fix:** written in `PayoutBatch` and `PayoutSplitBatch` inside the payout transaction; removed from the dispatcher. **Verified:** Go test against a real database: two payout calls produce exactly one row, carrying the batch's net. **Existing data:** historical duplicates and missing rows need a one-off reconciliation (proposed: keep the earliest row per batch, back-fill paid batches that have none). Not run.

F78 — Closing an unfinished business day strands captures

Symptom: an admin or CLI `run-batch` for today (or a future date) closed and paid the batch, and every capture later that day was never settled. **Root cause:** no guard; a closed batch makes the re-run a no-op, and the next day's window does not reach back. **Fix:** `ErrBusinessDayNotComplete` in `CloseBatch` and `RunDailyBatch`; the internal API returns 400 `business_day_not_complete`. The daemon is unaffected (it always closes yesterday). **Verified:** today and tomorrow are refused and leave no batch row; yesterday closes. **Existing data:** query for capture events with `settlement_batch_id IS NULL` older than one day to find any stranded funds.

F79 — Analytics rebuild drops refunds and double-counts

Symptom: after **Rebuild analytics**, refund totals were zero, and captures still waiting in the reporting worker's backlog were counted twice. **Root cause:** a second implementation that rebuilt from `transaction_events` instead of the outbox the worker consumes. **Fix:** rebuild from the outbox up to the worker's watermark, under the TRUNCATE lock, with the worker's own buckets. **Verified:** refunds kept; events past the watermark are not counted. **Existing data:** re-run the rebuild once.

F80 — Hold expiry deletes every hold

Symptom: on a night with no expired holds, `scripts/expire_holds.ts` ran `DELETE FROM holds` with no WHERE clause. **Root cause:** `buildWhere` silently dropped an empty array filter. **Fix:** an empty list now means `WHERE FALSE`; the script skips the delete when nothing matched, and runs in one transaction with `FOR UPDATE`. **Verified:** with one live hold and zero expired, nothing is deleted; with one expired, only that one. **Existing data:** holds already deleted cannot be restored from this table.

F81 — Refunded transactions crash the merchant dashboard

Symptom: the detail page of any refunded transaction, and the reconciliation page, failed to render. **Root cause:** the dashboard's `bhd()` called `toFils` without `allowNegative`, but refund timeline amounts are negative by design (migration 0044). **Fix:** signed formatting, as the admin console already does. **Verified:** `bhd(-4750)` returns `"-4.750 BHD"`.

F82 — Merchants logged out after 10 minutes

Root cause: OTP verify promoted the session to 12 h but never re-issued the cookie, which kept the pending session's 10-minute `maxAge`. **Fix:** re-issue the cookie, as admin verify does. **Verified:** the verify response sets `Max-Age=43200`.

F83, F84, F85, F87 — Bad input returned 500 instead of 4xx

Four separate inputs reached the database or Node unchecked: unknown profile-PATCH keys (`SET undefined = $2`); invalid transaction `from`/`to` (`"NaN-NaN…"` sent to Postgres, although the route documents 400); a KYC filename outside latin-1, e.g. Arabic (Node rejects the header value with `ERR_INVALID_CHAR`); and an unknown `businessType` (DB CHECK violation). **Fixes:** allow-list the patchable keys, validate dates, use an RFC 6266 `filename*` header, validate business types. **Verified:** each reproduced as a 500 first, then returns 400 or 200 after the fix.

F86 — Database published on all interfaces

Root cause: compose mapped `5432:5432` with the committed dev password, and the hosted box runs the same file. **Fix:** bound to `127.0.0.1`.

F88 — Capture replay double-credits a negotiated discount

Symptom/reproduction: merchant at 250 bps with a 10% discount: capture 100 fils, then 1,000,000 fils, then replay the first capture's idempotency key. The merchant was credited 2.500 BD for the second capture *twice*. **Root cause:** the guard compared "adjustments < capture events" and then adjusted the *latest* capture. A capture whose discount rounds to 0 never gets an adjustment, so the counts never catch up. **Fix:** match adjustments to the captures that are owed one, oldest first. **Verified:** the same reproduction now credits once. **Existing data:** look for transactions with more discount-adjustment groups than captures owed one.

F89 — Inbound acquirer signature checked over re-serialized JSON

Symptom: a correctly signed callback with whitespace or escaped unicode was rejected with 401. This is the incident log's "the incoming signature check has its own problems". **Root cause:** the HMAC was computed over `JSON.stringify(parsed body)`, not the bytes received. **Fix:** the callback route keeps the raw body and verifies that. **Verified:** a pretty-printed, escaped body signed over its raw bytes returns 200; a tampered body returns 401; malformed JSON returns 400.

F90, F91, F92 — Smaller correctness gaps

F90: credits accepted a future effective date, although the contract rejects it, so ledger legs were dated in a period that hasn't happened yet. Now dates after today's Bahrain date return 400. **F91:** split payouts never updated the lifetime payout estimate; fixed. **F92:** the admin console's default run-batch date was two days ago between 00:00 and 03:00 Bahrain time; it now defaults to the last completed Bahrain day.

Platform volume counter (item 73, partly fixed)

In Go, `current + int32(gross)` wrapped negative past 2,147,483,647 fils. That is only about 147,483 BD after the 2,000,000,000-fils volume tier switches on, so the 15 bps bonus would silently switch back off. The counter now stops at its maximum instead of wrapping; widening the INTEGER column still needs a DBA-approved migration.

Items 1–74 (first rounds), summary

Group	Items	Status
Auth, OTP, impersonation, admin roles, KYC, API keys	4–6, 12–15, 18–20, 54, 60–61	Fixed
Callbacks, webhooks, payout notices (FP-201, FP-240, 08-21)	7–9, 22, 55–57, 64	Fixed
Config and Docker (prod in dev mode, destructive seed on `up`)	10–11, 67	Fixed
Payment-link lock starving the pool (05-14 follow-up)	21	Fixed

Refunds / FP-300 end to end	30–38	Fixed (v1 501 stub kept by instruction)
Settlement: scheme residue, splits, paid-batch MDR, fee rates, daemon timing, test mode	42–46, 48, 62–63, 65–66	Fixed
Balances, reporting, reconciliation	39–41, 51–53, 69	Fixed
March sweep, CSV import, seed	23–29, 49–50, 59, 68	Fixed
Retained by explicit instruction or approval needed	1–3, 16–17, 47, 71–72, 74; 73 partly fixed	Left as is (see Decisions)
Open / no change needed	70 (commit-order outbox); 58 (jitter is correct)	Open / n/a

Root causes

Underlying cause	Findings it explains
Duplicate implementations that drifted apart. The same figure is derived twice and nothing reconciles the two.	Balance button vs CLI (52), analytics rebuild vs worker (F79), capture fee vs settlement fee (62–63, F88), disbursement record via webhooks (F77), fee display vs rate in effect (46)
Check-then-act without a lock or a unique constraint.	Concurrent refunds (31), duplicate payout notices (22, 64), OTP guess limit (5–6), duplicate imports (27–28), hold expiry SELECT/DELETE (F80)
Wrong unit or wrong clock. Two-decimal assumptions in a three-decimal currency; UTC where the business day is Asia/Bahrain.	CSV 10× (49), discount scaling (50), March window (23), daemon at 03:15 (65), reporting dates (39), admin default date (F92)
Trusting input. Client clock, re-serialized bodies, unchecked keys and enums.	Expired link paid via client clock (FP-187), unsigned or replayed callbacks (7–8), inbound signature (F89), F83–F87
Missing authorization at the route.	F75, F76, impersonation scope (12–15), bulk KYC (18–19), void role (54)
Process. Agent instruction files (`CLAUDE.md` , `AGENTS.md` , `.cursorrules`) state false facts: amounts in "cents", UTC settlement windows, "docs are truth". Locked tests pin the defects.	Explains why earlier AI-assisted changes kept reintroducing bugs (see section 6)

Decisions and open risks

Not changed	Why	Proposed safeguard
`Math.random()` for OTPs, invite codes, link slugs (1–3)	Explicit repository instruction	Switch to `crypto.randomInt` ; it's a one-line change once policy allows
Credit approval compares display names (16–17)	Explicit instruction	Compare `admin_user_id` (the handover's own rule)
INTEGER platform counter, REAL lifetime estimate (73–74)	Needs a DBA-approved migration	Migrate both to BIGINT fils; the clamp (73) prevents the tier flip meanwhile
Outbox read by id, not commit order (70)	Design change	Use the `xid8` / `pg_snapshot` query mentioned in the handover
Locked tests asserting old behaviour (7 API, 2 Go, 1 script)	Tests must not be edited	Replace them with tests that assert the ADRs

Historical data (F77 duplicates, F78 stranded captures, F88 double credits)	Fixes are forward-only; nothing was run on real data	Read-only detection queries first, then finance-approved corrections

Plausible, not confirmed:

- No private-address check on webhook URLs (SSRF): "send test event" reflects the response body.
- One malformed outbox event stops the reporting worker; it needs a dead-letter policy.
- Recalculate MDR re-prices at the flat `mdr_bps` and ignores fee schedules; the locked test name suggests this is intended.
- The capture route has no merchant role gate.
- Password login has no rate limit.

5. UI and UX changes

Change	Why it mattered	Before / after
Dashboard renders negative amounts (refund rows, reconciliation gap)	Refunded transactions and the reconciliation page went blank	**[screenshots: detail page of a refunded transaction]**
Session cookie lasts the session's 12 h	Merchants were signed out 10 minutes after login	**[screenshot / short recording]**
Admin run-batch defaults to the last completed Bahrain day	Around midnight it defaulted to two days ago	**[screenshots]**
KYC documents with Arabic filenames download	Download failed with 500	**[screenshots]**

6. AI usage

Setup. Claude Code (VS Code extension) with read and write access to the repository, a disposable Postgres 16 container (`filspay-test`) with one fresh database per verification run, and the Go and Node toolchains. Every fix was verified on an untouched export of `HEAD` plus only our patch, so concurrent edits in the shared working tree could not affect results. **[List your committed AI setup files here: agent instructions, custom commands, permission rules, helper scripts, and where they live.]** We deliberately did *not* follow the repository's own agent files where they contradict the ADRs: "amounts are cents", "settlement windows are UTC", "docs are the source of truth".

Excerpt 1 — F80, hold expiry deletes every hold

Prompt and context: a broad review prompt (review prior work, find new defects, fix, verify, don't edit tests) with the full repository, handover and ADRs. **AI answer (excerpt):**

```
Critical finding: scripts/expire_holds.ts runs `DELETE FROM holds` with no WHERE clause on
any night
with no expired holds, because buildWhere drops empty arrays. That wipes every hold, active
ones
included. It also deletes rows the handover says must only be flipped to `expired`.
```

How we verified: read `sqlFilter.buildWhere` (empty array → `continue`), then ran the patched script on a database with one live hold and zero expired holds: `matched 0 … deleted 0 row(s)`, and the live hold remained. With one expired hold, only that one was deleted. **What we changed:** empty list → `FALSE`; skip the delete when nothing matched; one transaction with `FOR UPDATE`. The AI kept the locked test's `DELETE … ANY($1)` text unchanged.

Excerpt 2 — F88, discount double credit

Context: in its first report the AI listed this as only "plausible but unconfirmed". When we asked "is everything finished?", it went back and reproduced it. **AI output (excerpt):**

```
statuses 201 200 200 after two captures: { n: 1, s: 2500 }
after replaying capture 1: { n: 2, s: 5000 }
Confirmed, and it's a real money defect. Replaying the first capture's idempotency key
```

```
credits the
merchant the second capture's 2.500 BD discount a second time.
```

Verified: after the fix, the same script printed `after replaying capture 1: { n: 1, s: 2500 }`, and the discount/refund/transaction suites passed apart from the one pre-existing stale assertion. **Lesson:** labelling something "plausible" is not the end of the job; we required a reproduction before accepting or rejecting each lead.

Excerpt 3 — the AI was wrong about its own verification

What happened: to test "our patch only", the AI generated a patch from a list of changed files and ran the suites on a clean copy. In zsh, `git diff -- $MINE` does not split the variable into separate paths, so the first patch was empty. A later attempt filtered a one-line file list with `grep -v`, which dropped every file. For a while, a "patched" copy contained only 2 of 16 fixes, while the AI described it as the full patch. **How it was noticed:** before building the submission, the AI compared every changed file in the clean copy against the working tree. The check printed `differs from live` for 16 files and the patch held only 8 diffs. **What we changed:** rebuilt the clean tree file by file, confirmed it was byte-identical to the patch result (`WT3_EQUALS_MINE`), and re-ran every suite and reproduction on it. A related catch: a previous AI-written inventory said `go test ./...` passed. The database-backed Go tests silently skip when `DATABASE_URL` is unset, and with a real database two tests fail.

Appendix

Test results — clean `HEAD` + our patch, fresh migrated database (0001–0079)

Check	Result	Notes
`git diff --check`, `npm run typecheck`, `go vet`	clean	
money / events / web	40/40, 4/4, 8/8	
API	355/362	7 locked tests assert pre-fix behaviour (4.75 BD = 475 fils, discount off the fee, fee display, exact jitter)
Reporting	7/11 combined, 11/11 alone	Shared-database contamination from API tests
Scripts	4/5	Locked test expects the UTC month; ADR-0007 requires the Bahrain month
Go	all packages ok except `ledger` (2)	`refresh_test.go` asserts the pre-ADR-0008 balance source (FP-203)

The same set fails on the unmodified baseline: our changes add no new failures. Logs are in `submission/evidence/`.